

SDLC та STLC. Методології розробки ПЗ.

Домашнє завдання

Beet Seed — відпрацюй навички на базовому рівні.

Склади порівняльну таблицю найбільш поширених методологій:

№	Назва методології	Сильні сторони	Слабкі сторони	Для якої галузі є доцільною
1	Waterfall	Чітка структура етапів, легке планування, добре підходить для фіксованих вимог	Негнучкий до змін, пізні тестування, ризик виявлення помилок наприкінці	Будівництво, виробництво..
2	V-model	Тестування на кожному етапі, зрозуміла документація	Дорогий, складний унесенні змін, потребує детального планування	Медицина, авіація, автомобільна промисловість
3	Spinal model	Орієнтація на ризики, гнучкість, раннє виявлення проблем	Складний в управлінні, дорогий, потребує досвідчену команду	Великі ІТ-проекти, оборонна сфера
4	Scrum	Швидка адаптація до змін, регулярний зворотний зв'язок, висока залученість команди	Потребує дисципліни, складно масштабувати, не підходить для фіксованих вимог	ІТ-стартапи, веб-розробка, мобільні додатки
5	Kanban	Візуалізація процесу, гнучкість, безперервне покращення	Відсутність чітких дедлайнів, складно прогнозувати терміни	Підтримка продуктів, сервісні команди
6	Extreme	Висока якість коду, часті релізи, тісна співпраця з замовником	Потребує високої кваліфікації, складно для великих команд	Розробка ПЗ з частими змінами вимог

Beet Sprout — детальніше заглибис в практику.

1. Виконай завдання попереднього рівня.

2. Напиши розгорнуті відповіді (0,5 - 1 сторінки тексту) на такі два питання:

На твою думку, чому з'явився Agile-маніфест?
Які проблеми він мав вирішити і чи це вдалося?
Agile-маніфест з'явився через проблеми, які масово виникали під час розробки програмного забезпечення за традиційними методологіями, такими як Waterfall або V-model. Ці методології передбачали детальне планування всього проекту наперед, фіксовані вимоги та довгий цикл розробки, у результаті чого замовник бачив готовий продукт лише наприкінці роботи. На практиці це часто призводило до ситуацій, коли продукт уже не відповідав реальним потребам бізнесу, вимоги змінювалися в процесі роботи, а виправлення помилок коштувало багато часу і ресурсів. Команди втрачали мотивацію, а компанії гроші.
Agile-маніфест мав на меті змінити сам підхід до розробки. Зробити його більш гнучким, орієнтованим на людину, постійну співпрацю із замовником та швидке отримання зворотного зв'язку. Основна ідея полягала в тому, що зміни є неминучими, тому процес повинен легко до них адаптуватися, а не намагатися жорстко дотримуватися плану, складеного на початку проекту.
Загалом Agile частково досяг своєї мети. Розробка стала швидшою, прозорішою, а якість взаємодії між командою та замовником покращилися. Продукти почали виходити на ринок раніше, а ризик створення непотрібного або застарілого рішення зменшився. Водночас у багатьох компаніях Agile був зведений до формального набору правил і мітингів, втративши свою початкову філософію гнучкості та довіри до команди. Тому можна сказати, що Agile вирішив значну частину проблем класичних методологій, але його ефективність безпосередньо залежить від того, наскільки правильно і усвідомлено він застосовується на практиці.

Mighty Beet — різнобічно опануй тематику уроку.

1. Виконай завдання двох попередніх рівнів.

2. Ти – засновник/ця стартапу і плануєш випустити на ринок мобільний застосунок для обміну світлинами котиків.

Яку методологію ти обереш для процесу розробки і чому? Відповідь текстово обґрунтуй.
Так як це буде стартап, а вище вже в таблиці робила порівняння і визначилася що для стартапів найкраще підходить Scrum, тож я б використала саме цю методологію. Так як розробка мобільного застосунку для обміну світлинами котиків передбачає постійні зміни вимог і активну взаємодію з користувачами. На початковому етапі стартапу складно точно передбачити, які функції будуть найбільш популярними, тому важливо мати можливість швидко тестувати ідеї та вносити зміни.
Scrum дозволяє розробляти продукт спринтами, регулярно випускати нові версії застосунку та збирати відгуки користувачів. Це зменшує ризик створити продукт, який не відповідає очікуванням ринку, і допомагає поступово покращувати його на основі реального досвіду користувачів.
Якби не Scrum то на мою думку ще підходить Kanban. Я вважаю що дана методологія теж гарно підійде для даної ситуації. Насправді все залежитиме від розміру та досвіду команди, вимог до якості коду і необхідної швидкості виходу на ринок.